

CS3 Capstone Projects
Stage 2: Planning and Modelling

NXXLET001 MKHMAX003 BLYKEN007

Use Cases

for

<UAV Path Visualizer >

Version 1.0 approved

Prepared by <NXXLET001, MKHMAX003, BLYKEN007>

<Capstone Project>

<08/08/2025>

1. Use Case

- **Use Case ID:0001**
- **Use Case Name: First-Time Scene Loading and Exploration**
- **Actor: A UAV Operator**
- **Description: UAV operator loads a 3D model into application to display scene for path planning.**
- **Trigger: The operator needs to view a 3D structure before analyzing the flight path.**
- **Preconditions: The UAV path visualiser application is running.**
- **Postconditions: 3D model is rendered in the main viewport.**

Flow:

A UAV operator has just received a 3D model of some cultural heritage site that needs surveying. They perform an action that allows them to load a 3D model and select the valid file. The system loads the model, from a default view (e.g a birds eye perspective,) and displays the 3D structure on a reference ground plane, with axes shown in the corner for orientation. The operator can manipulate the camera scene via defined inputs.

- **Alternative Path (Invalid File):** The operator selects an unsupported file. The system rejects and displays an error message and displays the supported file types. The use case can continue.
- **Alternative Path (Corrupt Model file):** The system is unable to parse file. The system does not crash but will display an error message indicating the file is corrupt and cannot be loaded. The use case terminates.
- **Alternative Path (Missing material file):** The system successfully parses the model file but cannot locate associated material or texture image files referenced within it. The system will render the 3D model using default, untextured material. The system displays a non-blocking warning message and use case continues.

2. Use Case

- **Use Case ID:0002**
- **Use Case Name: Visualizing and understanding a flight path**
- **Actor: A UAV Operator**
- **Description: The operator inspects a specific point on the flight path to see the drone's camera perspective**
- **Trigger: The operator needs to evaluate a flight path.**
- **Preconditions: A 3D model has successfully been loaded into the scene.**
- **Postconditions: Flight path is rendered in the scene as a navigable 3D curve with orientation markers.**

Flow:

Now the scene is loaded, the operator wants to see proposed flight path, they initiate action to load a trajectory file. Operator will select valid trajectory file. The system will process the file and output flight path. Objects are shown along the line showing where the drone took a picture and the orientation of camera will be pointing.

- **Alternative Path (Corrupt File):** The system attempts to parse the file but encounters a structural error or missing data. The system will display a specific error message that will detail where the error or missing data is. The use case terminates
- **Alternative Path (Trajectory Scene mismatch):** The system successfully parses the trajectory file but detects metadata in file does not match with currently loaded 3D model. The system displays a warning message and use case terminates.
- **Alternative Path (Path is outside visible bounds):** The system renders the path, but its coordinates outside the current camera's view. The system detects that the newly added object is not visible and asks the user to automatically adjust to fit the path. The operator confirms, the system automatically adjusts camera's position and zoom to frame both 3D model and trajectory, use case continues.

3. Use Case

- **Use Case ID:0003**
- **Use Case Name: In-Depth Way point inspection with First-Person View (FPV).**
- **Actor: A UAV Operator**
- **Trigger: The operator needs to verify the camera's line of sight at a specific waypoint.**
- **Preconditions: A trajectory has been successfully loaded.**
- **Postconditions: FPV for a selected waypoint is displayed**

Flow:

The operator selects a waypoint marker either in 3D scene or from waypoint list panel. It now shows their exactly what the drone's camera would see from that specific point.

- **Alternative Path (No WayPoint selected):** The FPV window displays a default state.use case continues.
- **Alternative Path (User deselects waypoint):** The system removes the highlighting from waypoint marker. The FPV window reverts to its default state. Use case continues.

4. Use Case

- **Use Case ID:0004**
- **Use Case Name: Comparing Flight Paths.**
- **Actor: A UAV Operator**
- **Trigger: The operator needs to survey multiple path options to evaluate.**
- **Preconditions: At least one trajectory has been successfully loaded.**
- **Postconditions: Two or more path are displayed in the scene and their comparative metrics are shown in the UI.**

Description:

The UAV operator performs action to load another trajectory file. Operator selects second valid trajectory file. Comparative metrics of paths are displayed.

- **Alternative Path (Duplicate Path):** The system detects the duplicate and displays an informational message(Flight path is loaded you want to continue?). User confirms, use case continues.
- **Alternative Path (Metric Calculation fails):** The system encounters invalid data for one of the paths. The system displays “N/A” or error in the comparison table for affected path. System logs error and does not crash use case continues.

5.

Use Case

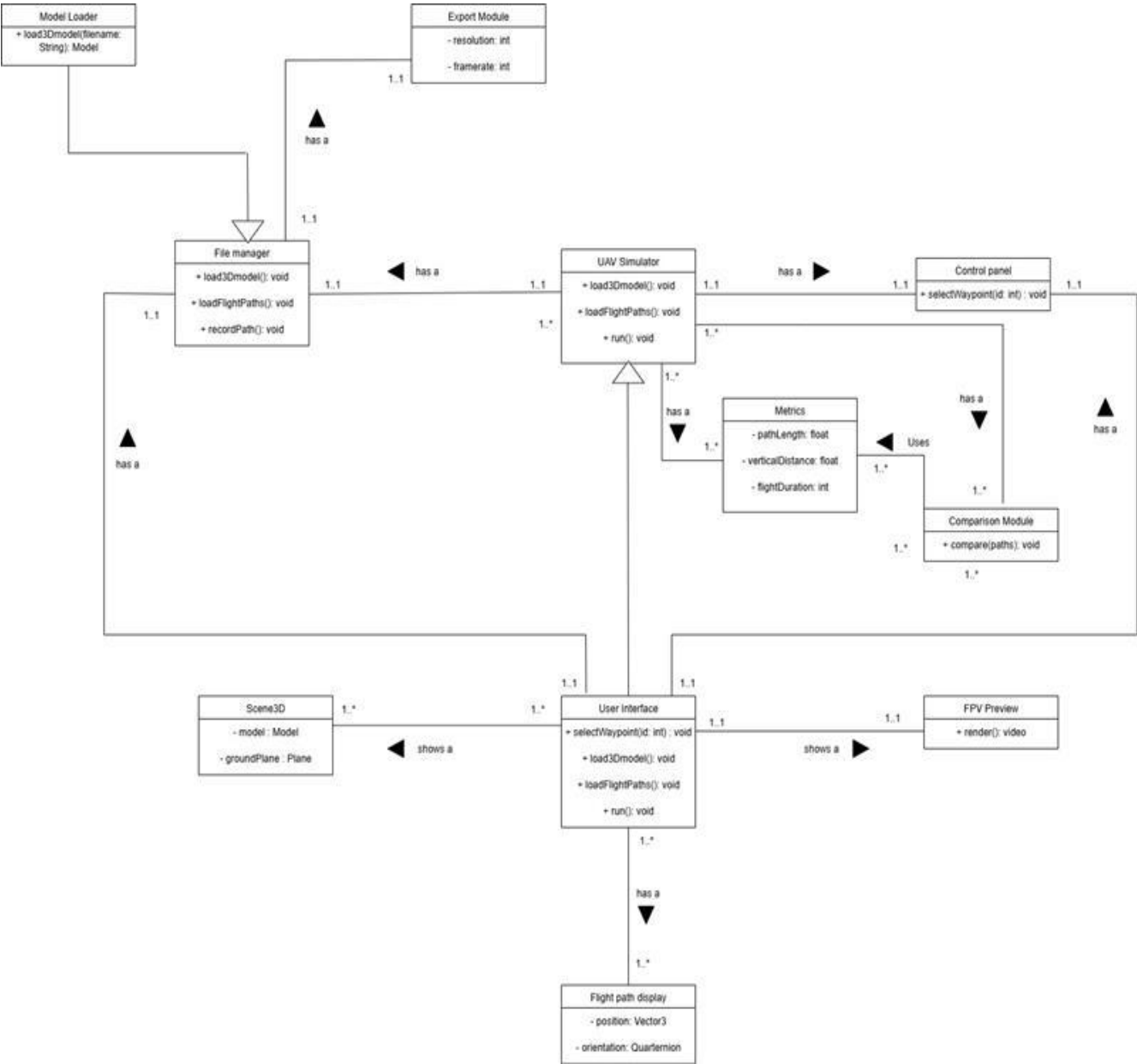
- **Use Case ID:0005**
- **Use Case Name: Create a video of a simulation**
- **Actor: A UAV Operator**
- **Trigger: The operator needs to share a dynamic preview of proposed mission.**
- **Preconditions: A trajectory has been successfully loaded.**
- **Postconditions: A video file of flight simulation is saved to users computer.**

Description:

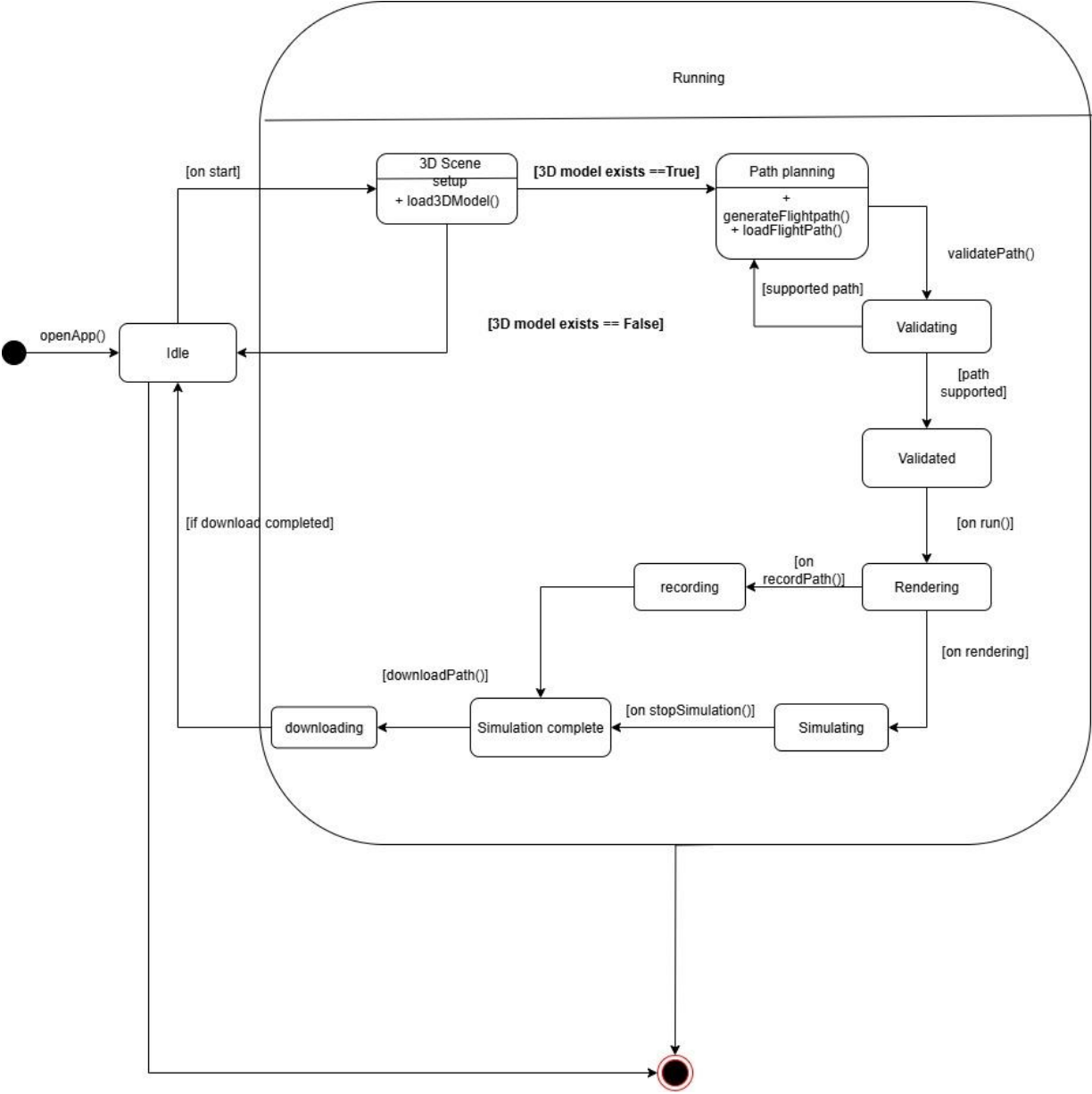
The operator selects a flight path to simulate. The operator sets the desired video resolution and framerate and initiates the recording. The recording completes

- **Alternative Path (Recording Fails):**During recording, the system detects system insufficient disk space. The system immediately stops recording, deletes created file and displays error message to notify operator to free up required disk space for recording. Use case terminates.
- **Alternative Path (User cancels recording):** The operators performs action to cancel recording. The system displays a confirmation message, the user confirms and the recording process stops. The system deletes the partially recorded file. Use case terminates.
- **Alternative Path (No trajectory Selected for recording):** The operator initiates the recording action without first selecting trajectory. The system displays a prompt. The use case terminates and returns the user to main interface.

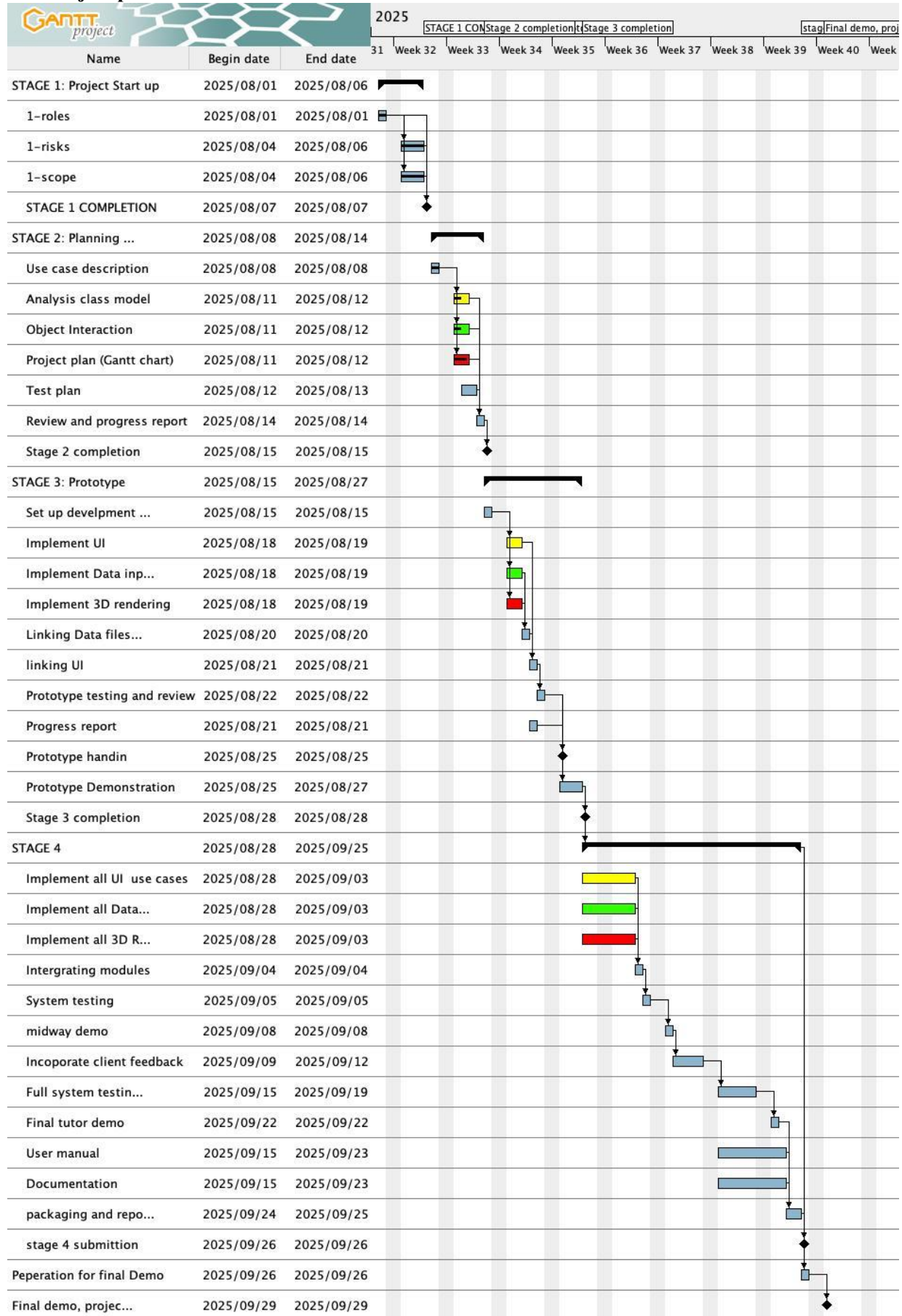
1. Analysis Class model [10]



2. Interaction Diagram or State Diagram [10]



3. Project plan



4. A preliminary test plan [10 points as specified below]

You must plan and perform unit testing, failure to do so will result in zero [0] points for your test plan.

- **Test strategy:** Describe your test strategy [1]
 1. Unit testing. Unit testing will be implement in our project to ensure that each unit of the system is functional. These will consist of a unit test for data handling, a unit testing for the user interface, a unit testing for the camera movements, a unit testing for 3D rendering and a unit testing for file download.
 2. Integration testing: Since we will be working on different modules of the code separately we will perform integration testing to ensure these modules are integrated together seamlessly.
 3. Regression testing: Regression testing will be performed after each integration to ensure a consistent flow of the system
 4. System Testing: This testing will be perform lastly to ensure that the system behave as it should and it meets the client and users requirements.
- **Automated tools:** Describe what automated tools you intend to use to catch potential errors, what kind of errors these tools can detect and when in the development cycle you use these tools. If you do not use automated tools, give valid reasons why you cannot use such tools, or why you do not need them [2]
 1. We will use **Atheris** to test for fuzzing. This will help in testing for error handling and to see if the program misbehave. It help in performing security checks. It will be used during implementation of data handling component to prevent errors and ensure proper pipeline of data.
 2. Unittest: This will be used for automating unit testing, It helps in catching of logical error in the code or some broken functionality. I will be used through out the development stage after each component has been completed.
- **Unit tests:** Describe how you will create unit tests [2], how you will measure unit test coverage [2], list the unit testing framework you intend to use [1], and specify who is in charge of ensuring test coverage and what actions that person will take to remedy insufficient coverage [1]

Each module will have its own test file to ensure that code can be tested even if they depend on components not yet implemented. We will achieve this by using Mock from the unittest library. The will be a unit test for 3D rendering (test_3D_rendering.py), a test for data handling(reading and writing) test_data_handling.py. We use correct data and invalid data to see how the system will behave in each case.

We will use <https://coverage.readthedocs.io/en/7.9.2/> for coverage which is suggested in lectures, we may also use coverage.py from the pytest library. Unittest will also be used for unit testing because it is built in in python and the is no need to download extra libraries.

The project leader will also be responsible for testing. The project leader knows the overall scope of the project and will ensure coverage report after each sprint, assign uncovered test cases do the developers and ensure that test coverage is sufficient before integration.

- **Additional actions:** Describe one or more additional planned actions to detect, reduce, and eliminate bugs [1]
 1. Gitlab will be used for version control with sufficient commits to make sure it is easier to go back to previous working code.
 2. There will be thorough code review before merging code from branch into the main branch.
 3. Constant meeting with the tutor will be arranged to ensure that usability and functional requirement are met.

Test Case	Inputs	Behaviour Tested	Expected Outcomes
Application Launch	User starts the application	Application start-up process	GUI loads with 3D viewport, control panel, and empty scene
Load correct 3D Model	.obj file	Model loading function	Model appears in viewport, textures applied correctly
Load Trajectory File	.JSON trajectory file with waypoints	Trajectory parser and path rendering	Spline path rendered in 3D view with waypoint markers
Compare Two Trajectories	Load two .JSON files	Multi-path rendering & metric calculation	Both paths visible, metrics shown side-by-side
Invalid Model File	.pdf file	Error handling for unsupported file types	Error dialog: "Unsupported model format, please enter the correct file"
Invalid Trajectory File	Corrupt .JSON	Parser error handling	Error dialog: "Invalid trajectory file"